Red Hat OpenShift AI Self-Managed 3.4

Working on projects

Organize your work in projects and workbenches, create and collaborate on notebooks, train and deploy models, configure model servers, and implement pipelines

Last Updated: 2026-06-16

Red Hat OpenShift AI Self-Managed 3.4 Working on projects

Organize your work in projects and workbenches, create and collaborate on notebooks, train and deploy models, configure model servers, and implement pipelines

Legal Notice

Abstract

Organize your work in projects and workbenches, create and collaborate on notebooks, train and deploy models, configure model servers, and implement pipelines.

Table of Contents

PREFACE

As a data scientist, you can organize your data science work into a single project. A project in OpenShift AI can consist of the following components:

Workbenches

Creating a workbench allows you to work with models in your preferred IDE, such as JupyterLab.

Cluster storage

For projects that require data retention, you can add cluster storage to the project.

Connections

Adding a connection to your project allows you to connect data inputs to your workbenches.

Pipelines

Standardize and automate machine learning workflows to enable you to further enhance and deploy your data science models.

Models and model servers

Deploy a trained data science model to serve intelligent applications. Your model is deployed with an endpoint that allows applications to send requests to the model.

Bias metrics for models

Creating bias metrics allows you to monitor your machine learning models for bias.

CHAPTER 1. USING PROJECTS

1.1. CREATING A PROJECT

To implement a data science workflow, you must create a project. In OpenShift, a project is a Kubernetes namespace with additional annotations, and is the main way that you can manage user access to resources. A project organizes your data science work in one place and also allows you to collaborate with other developers and data scientists in your organization.

Within a project, you can add the following functionality:

- Connections so that you can access data without having to hardcode information like endpoints or credentials.
- Workbenches for working with and processing data, and for developing models.
- Deployed models so that you can test them and then integrate them into intelligent applications. Deploying a model makes it available as a service that you can access by using an API.
- Pipelines for automating your ML workflow.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have the appropriate roles and permissions to create projects.

Procedure

1. From the OpenShift AI dashboard, select **Projects**.
   The **Projects** page shows a list of projects that you can access. For each user-requested project in the list, the **Name** column shows the project display name, the user who requested the project, and the project description.
2. Click **Create project**.
3. In the **Create project** dialog, update the **Name** field to enter a unique display name for your project.
4. Optional: If you want to change the default resource name for your project, click **Edit resource name**.
   The resource name is what your resource is labeled in OpenShift. Valid characters include lowercase letters, numbers, and hyphens (-). The resource name cannot exceed 30 characters, and it must start with a letter and end with a letter or number.

   Note: You cannot change the resource name after the project is created. You can edit only the display name and the description.
5. Optional: In the **Description** field, provide a project description.
6. Click **Create**.

Verification

- A project details page opens. From this page, you can add connections, create workbenches, configure pipelines, and deploy models.

1.2. UPDATING A PROJECT

You can update the project details by changing the project name and description.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the action menu (⋮) beside the project whose details you want to update and click **Edit project**.
   The **Edit project** dialog opens.
3. Optional: Edit the **Name** field to change the display name for your project.
4. Optional: Edit the **Description** field to change the description of your project.
5. Click **Update**.

Verification

- You can see the updated project details on the **Projects** page.

1.3. DELETING A PROJECT

You can delete projects so that they do not appear on the OpenShift AI **Projects** page when you no longer want to use them.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the action menu (⋮) beside the project that you want to delete and then click **Delete project**.
   The **Delete project** dialog opens.
3. Enter the project name in the text field to confirm that you intend to delete it.

4. Click **Delete project**.

Verification

- The project that you deleted is no longer displayed on the **Projects** page.
- Deleting a project deletes any associated workbenches, AI pipelines, cluster storage, and connections. This data is permanently deleted and is not recoverable.

CHAPTER 2. USING PROJECT WORKBENCHES

2.1. CREATING A WORKBENCH AND SELECTING AN IDE

A workbench is an isolated area where you can examine and work with ML models. You can also work with data and run programs, for example to prepare and clean data. While a workbench is not required if, for example, you only want to service an existing model, one is needed for most data science workflow tasks, such as writing code to process data or training a model.

When you create a workbench, you specify an image (an IDE, packages, and other dependencies). Supported IDEs include JupyterLab, code-server, and RStudio (Technology Preview).

The IDEs are based on a server-client architecture. Each IDE provides a server that runs in a container on the OpenShift cluster, while the user interface (the client) is displayed in your web browser. For example, the Jupyter workbench runs in a container on the Red Hat OpenShift cluster. The client is the JupyterLab interface that opens in your web browser on your local computer. All of the commands that you enter in JupyterLab are executed by the workbench. Similarly, other IDEs like code-server or RStudio Server provide a server that runs in a container on the OpenShift cluster, while the user interface is displayed in your web browser. This architecture allows you to interact through your local computer in a browser environment, while all processing occurs on the cluster. The cluster provides the benefits of larger available resources and security because the data being processed never leaves the cluster.

In a workbench, you can also configure connections (to access external data for training models and to save models so that you can deploy them) and cluster storage (for persisting data). Workbenches within the same project can share models and data through object storage with the AI pipelines and model servers.

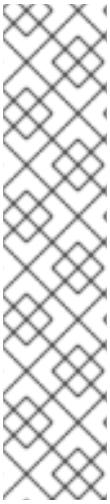
For projects that require data retention, you can add container storage to the workbench you are creating.

Within a project, you can create multiple workbenches. When to create a new workbench depends on considerations, such as the following:

- The workbench configuration (for example, CPU, RAM, or IDE). If you want to avoid editing the configuration of an existing workbench's configuration to accommodate a new task, you can create a new workbench instead.
- Separation of tasks or activities. For example, you might want to use one workbench for your Large Language Models (LLM) experimentation activities, another workbench dedicated to a demo, and another workbench for testing.

2.1.1. About workbench images

A workbench image is preinstalled with tools and libraries for model development. You can use the provided images, or an OpenShift AI administrator can create custom images tailored to your needs.



NOTE

As of Red Hat OpenShift AI version 2.25, the LLM compressor library is generally available and integrated into supported Red Hat OpenShift AI workbench images and AI pipelines.

This library provides a supported, integrated method for optimizing large language models to improve inference, particularly for deployment on vLLM, without leaving your Red Hat OpenShift AI environment. You can run model compression as an interactive notebook task or as a batch job in your AI pipeline to significantly reduce your hardware costs and improve the inference speeds of your generative AI workloads.

For more information about model compression, see Supported model compression workflows.

To provide a consistent, stable platform for your model development, many provided workbench images contain the same version of Python. Most workbench images available on OpenShift AI are pre-built and ready for you to use immediately after OpenShift AI is installed or upgraded.

For information about Red Hat support of workbench images and packages, see Supported Configurations for 3.x.

The following table lists the workbench images that are installed with Red Hat OpenShift AI by default.

If the preinstalled packages that are provided in these images are not sufficient for your use case, you have the following options:

- Install additional libraries after launching a default image. This option is good if you want to add libraries on an ad hoc basis as you develop models. However, it can be challenging to manage the dependencies of installed libraries and your changes are not saved when the workbench restarts.
- Create a custom image that includes the additional libraries or packages. For more information, see Creating custom workbench images.

IMPORTANT

Workbench images denoted with **(Technology Preview)** in this table are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using Technology Preview features in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope.

Table 2.1. Default workbench images

Image name	Description
CUDA	If you are working with compute-intensive data science models that require GPU support, use the Compute Unified Device Architecture (CUDA) workbench image to gain access to the NVIDIA CUDA Toolkit. Using this toolkit, you can optimize your work by using GPU-accelerated libraries and optimization tools.

Image name	Description
Standard Data Science	Use the Standard Data Science workbench image for models that do not require TensorFlow or PyTorch. This image contains commonly-used libraries to assist you in developing your machine learning models.
TensorFlow	TensorFlow is an open source platform for machine learning. With TensorFlow, you can build, train and deploy your machine learning models. TensorFlow contains advanced data visualization features, such as computational graph visualizations. It also allows you to easily monitor and track the progress of your models.
PyTorch	PyTorch is an open source machine learning library optimized for deep learning. If you are working with computer vision or natural language processing models, use the Pytorch workbench image.
Minimal Python	If you do not require advanced machine learning features, or additional resources for compute-intensive data science work, you can use the Minimal Python image to develop your models.
TrustyAI	Use the TrustyAI workbench image to leverage your data science work with model explainability, tracing, and accountability, and runtime monitoring. See the TrustyAI Explainability repository for some example Jupyter notebooks.
code-server	With the code-server workbench image, you can customize your workbench environment to meet your needs using a variety of extensions to add new languages, themes, debuggers, and connect to additional services. Enhance the efficiency of your data science work with syntax highlighting, auto-indentation, and bracket matching, as well as an automatic task runner for seamless automation. For more information, see code-server in GitHub. NOTE: Elyra-based pipelines are not available with the code-server workbench image.
RStudio Server (Technology preview)	Use the RStudio Server workbench image to access the RStudio IDE, an integrated development environment for R, a programming language for statistical computing and graphics. For more information, see the RStudio Server site. To use the **RStudio Server** workbench image, you must first build it by creating a secret and triggering the BuildConfig, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, seeBuilding the RStudio Server workbench images.  **IMPORTANT** **Disclaimer:** Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through https://rstudio.org/ and is subject to RStudio licensing terms. Review the licensing terms before you use this sample workbench.

Image name	Description
CUDA - RStudio Server (Technology Preview)	Use the CUDA - RStudio Server workbench image to access the RStudio IDE and NVIDIA CUDA Toolkit. RStudio is an integrated development environment for R, a programming language for statistical computing and graphics. With the NVIDIA CUDA toolkit, you can optimize your work using GPU-accelerated libraries and optimization tools. For more information, see the RStudio Server site. To use the **CUDA - RStudio Server** workbench image, you must first build it by creating a secret and triggering the BuildConfig, and then enable it in the OpenShift AI UI by editing the **cuda-rstudio-rhel9** image stream. For more information, seeBuilding the RStudio Server workbench images.  **IMPORTANT** **Disclaimer:** Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through https://rstudio.org/ and is subject to RStudio licensing terms. Review the licensing terms before you use this sample workbench. The **CUDA - RStudio Server** workbench image contains NVIDIA CUDA technology. CUDA licensing information is available at https://docs.nvidia.com/cuda/. Review the licensing terms before you use this sample workbench.
ROCm	Use the ROCm workbench image to run AI and machine learning workloads on AMD GPUs in OpenShift AI. It includes ROCm libraries and tools optimized for high-performance GPU acceleration, supporting custom AI workflows and data processing tasks. Use this image integrating additional frameworks or dependencies tailored to your specific AI development needs.
ROCm-PyTorch	Use the ROCm-PyTorch workbench image to run PyTorch workloads on AMD GPUs in OpenShift AI. It includes ROCm-accelerated PyTorch libraries, enabling efficient deep learning training, inference, and experimentation. This image is designed for data scientists working with PyTorch-based workflows, offering integration with GPU scheduling.
ROCm-TensorFlow	Use the ROCm-TensorFlow workbench image to run TensorFlow workloads on AMD GPUs in OpenShift AI. It includes ROCm-accelerated TensorFlow libraries to support high-performance deep learning model training and inference. This image simplifies TensorFlow development on AMD GPUs and integrates with OpenShift AI for resource scaling and management.

2.1.2. Creating a workbench

When you create a workbench, you specify an image (an IDE, packages, and other dependencies). You can also configure connections, cluster storage, and add container storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.

- You have created a project.
- If you created a Simple Storage Service (S3) account outside of Red Hat OpenShift AI and you want to create connections to your existing S3 storage buckets, you have the following credential information for the storage buckets:
  - Endpoint URL
  - Access key
  - Secret key
  - Region
  - Bucket name

  For more information, see Working with data in an S3-compatible object store .

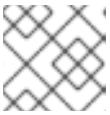
Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to add the workbench to.
   A project details page opens.
3. Click the **Workbenches** tab.
4. Click **Create workbench**.
   The **Create workbench** page opens.
5. In the **Name** field, enter a unique name for your workbench.
6. Optional: If you want to change the default resource name for your workbench, click **Edit resource name**.
   The resource name is what your resource is labeled in OpenShift. Valid characters include lowercase letters, numbers, and hyphens (-). The resource name cannot exceed 30 characters, and it must start with a letter and end with a letter or number.

   Note: You cannot change the resource name after the workbench is created. You can edit only the display name and the description.
7. Optional: In the **Description** field, enter a description for your workbench.
8. In the **Workbench image** section, complete the fields to specify the workbench image to use with your workbench.
   From the **Image selection** list, select a workbench image that suits your use case. A workbench image includes an IDE and Python packages (reusable code). If project-scoped images exist, the **Image selection** list includes subheadings to distinguish between global images and project-scoped images.

   Optionally, click **View package information** to view a list of packages that are included in the image that you selected.

   If the workbench image has multiple versions available, select the workbench image version to use from the **Version selection** list. To use the latest package versions, Red Hat recommends that you use the most recently added image.



NOTE

You can change the workbench image after you create the workbench.

9. In the **Deployment size** section, select one of the following options, depending on whether the hardware profiles feature is enabled.

IMPORTANT

The hardware profiles feature is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

- If the hardware profiles feature is not enabled:

    a. From the **Container size** list, select the appropriate size for the size of the model that you want to train or tune.
    For example, to run the example fine-tuning job described in Fine-tuning a model by using Kubeflow Training, select **Medium**.

    b. From the **Hardware profile** list, select a suitable hardware profile for your workbench.
    If project-scoped hardware profiles exist, the **Hardware profile** list includes subheadings to distinguish between global hardware profiles and project-scoped hardware profiles.

    The hardware profile specifies the number of CPUs and the amount of memory allocated to the container, setting the guaranteed minimum (request) and maximum (limit) for both.

    c. If you want to change the default values, click **Customize resource requests and limit** and enter new minimum (request) and maximum (limit) values.

10. Optional: In the **Environment variables** section, select and specify values for any environment variables.
Setting environment variables during the workbench configuration helps you save time later because you do not need to define them in the body of your workbenches, or with the IDE command line interface.

    If you are using S3-compatible storage, add these recommended environment variables:

    - **AWS_ACCESS_KEY_ID** specifies your Access Key ID for Amazon Web Services.
    - **AWS_SECRET_ACCESS_KEY** specifies your Secret access key for the account specified in **AWS_ACCESS_KEY_ID**.

IMPORTANT

Configure credentials to access only the S3 resources for this project. Sharing AWS credentials between projects can lead to compromised credentials and unauthorized access to data. Configure IAM policies that grant only the minimum permissions required for the specific S3 bucket.

OpenShift AI stores the credentials as Kubernetes secrets in a protected namespace if you select **Secret** when you add the variable.

11. In the **Cluster storage** section, configure the storage for your workbench. Select one of the following options:
    - **Create new persistent storage** to create storage that is retained after you shut down your workbench. Complete the relevant fields to define the storage:
        a. Enter a **name** for the cluster storage.
        b. Enter a **description** for the cluster storage.
        c. Select a **storage class** for the cluster storage.

        NOTE

        You cannot change the storage class after you add the cluster storage to the workbench.

        d. For storage classes that support multiple access modes, select an **Access mode** to define how the volume can be accessed. For more information, see About persistent storage.
        Only the access modes that have been enabled for the storage class by your cluster and OpenShift AI administrators are visible.
        e. Under **Persistent storage size**, enter a new size in gibibytes or mebibytes.
    - **Use existing persistent storage** to reuse existing storage and select the storage from the **Persistent storage** list.
12. Optional: You can add a connection to your workbench. A connection is a resource that contains the configuration parameters needed to connect to a data source or sink, such as an object storage bucket. You can use storage buckets for storing data, models, and pipeline artifacts. You can also use a connection to specify the location of a model that you want to deploy.
    In the **Connections** section, use an existing connection or create a new connection:
    - Use an existing connection as follows:
        a. Click **Attach existing connections**.
        b. From the **Connection** list, select a connection that you previously defined.
    - Create a new connection as follows:
        a. Click **Create connection**. The **Add connection** dialog opens.
        b. From the **Connection type** drop-down list, select the type of connection. The **Connection details** section is displayed.

c. If you selected **S3 compatible object storage** in the preceding step, configure the connection details:

i. In the **Connection name** field, enter a unique name for the connection.

ii. Optional: In the **Description** field, enter a description for the connection.

iii. In the **Access key** field, enter the access key ID for the S3-compatible object storage provider.

iv. In the **Secret key** field, enter the secret access key for the S3-compatible object storage account that you specified.

v. In the **Endpoint** field, enter the endpoint of your S3-compatible object storage bucket.

vi. In the **Region** field, enter the default region of your S3-compatible object storage account.

vii. In the **Bucket** field, enter the name of your S3-compatible object storage bucket.

viii. Click **Create**.

d. If you selected **URI** in the preceding step, configure the connection details:

i. In the **Connection name** field, enter a unique name for the connection.

ii. Optional: In the **Description** field, enter a description for the connection.

iii. In the **URI** field, enter the Uniform Resource Identifier (URI).

iv. Click **Create**.

13. Click **Create workbench**.

Verification

- The workbench that you created is visible on the **Workbenches** tab for the project.
- Any cluster storage that you associated with the workbench during the creation process is displayed on the **Cluster storage** tab for the project.
- The **Status** column on the **Workbenches** tab displays a status of **Starting** when the workbench server is starting, and **Running** when the workbench has successfully started.
- Optional: Click the open icon () to open the IDE in a new window.

2.2. STARTING A WORKBENCH

You can manually start a project's workbench from the **Workbenches** tab on the project details page. By default, workbenches start immediately after you create them.

Prerequisites

- You have logged in to Red Hat OpenShift AI.

- You have created a project that contains a workbench.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project whose workbench you want to start.
   A project details page opens.
3. Click the **Workbenches** tab.
4. In the **Status** column for the workbench that you want to start, click **Start**.
   The **Status** column changes from **Stopped** to **Starting** when the workbench server is starting, and then to **Running** when the workbench has successfully started. * Optional: Click the open icon () to open the IDE in a new window.

Verification

- The workbench that you started is displayed on the **Workbenches** tab for the project, with the status of **Running**.

2.3. UPDATING A PROJECT WORKBENCH

If your data science work requires you to change your workbench image, container size, or identifying information, you can update the properties of your project's workbench. If you require extra power for use with large datasets, you can assign accelerators to your workbench to optimize performance.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that has a workbench.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project whose workbench you want to update.
   A project details page opens.
3. Click the **Workbenches** tab.
4. Click the action menu (⋮) beside the workbench that you want to update and then click **Edit workbench**.
   The **Edit workbench** page opens.
5. Update any of the workbench properties and then click **Update workbench**.

Verification

- The workbench that you updated is displayed on the **Workbenches** tab for the project.

2.4. DELETING A WORKBENCH FROM A PROJECT

You can delete workbenches from your projects to help you remove Jupyter notebooks that are no longer relevant to your work.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project with a workbench.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to delete the workbench from.
   A project details page opens.
3. Click the **Workbenches** tab.
4. Click the action menu (⋮) beside the workbench that you want to delete and then click **Delete workbench**.
   The **Delete workbench** dialog opens.
5. Enter the name of the workbench in the text field to confirm that you intend to delete it.
6. Click **Delete workbench**.

Verification

- The workbench that you deleted is no longer displayed on the **Workbenches** tab for the project.
- The custom resource (CR) associated with the workbench's Jupyter notebook is deleted.

CHAPTER 3. USING CONNECTIONS

3.1. ADDING A CONNECTION TO YOUR PROJECT

You can enhance your project by adding a connection that contains the configuration parameters needed to connect to a data source or sink.

When you want to work with a very large data sets, you can store your data in an Open Container Initiative (OCI)-compliant registry, S3-compatible object storage bucket, or a URI-based repository, so that you do not fill up your local storage. You also have the option of associating the connection with an existing workbench that does not already have a connection.

Prerequisites

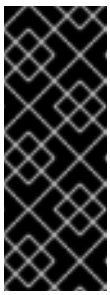
- You have logged in to Red Hat OpenShift AI.
- You have created a project that you can add a connection to.
- You have access to S3-compatible object storage, URI-based repository, or OCI-compliant registry.
- If you intend to add the connection to an existing workbench, you have saved any data in the workbench to avoid losing work.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to add a connection to.
   A project details page opens.
3. Click the **Connections** tab.
4. Click **Add connection**.
5. In the **Add connection** modal, select a **Connection type**. The **OCI-compliant registry**, **S3 compatible object storage**, and **URI** options are pre-installed connection types. Additional options might be available if your OpenShift AI administrator added them.
   The **Add connection** form opens with fields specific to the connection type that you selected.
6. Enter a unique name for the connection.
   A resource name is generated based on the name of the connection. A resource name is the label for the underlying resource in OpenShift.
7. Optional: Edit the default resource name. Note that you cannot change the resource name after you create the connection.
8. Optional: Provide a description of the connection.
9. Complete the form depending on the connection type that you selected. For example:
   a. If you selected **S3 compatible object storage** as the connection type, configure the connection details:

i. In the **Access key** field, enter the access key ID for the S3-compatible object storage provider.

ii. In the **Secret key** field, enter the secret access key for the S3-compatible object storage account that you specified.



IMPORTANT

To maintain security boundaries, use unique AWS credentials for each project. Do not share AWS access keys or secret keys across multiple projects. Configure IAM roles and users with policies that grant access only to the specific S3 bucket for the project. Using shared credentials with broad permissions violates the principle of least privilege and can allow unauthorized cross-project data access.

iii. In the **Endpoint** field, enter the endpoint of your S3-compatible object storage bucket.

NOTE

Make sure to use the appropriate endpoint format. Improper formatting might cause connection errors or restrict access to storage resources. For more information about how to format object storage endpoints, see Overview of object storage endpoints.

iv. In the **Region** field, enter the default region of your S3-compatible object storage account.

v. In the **Bucket** field, enter the name of your S3-compatible object storage bucket.

vi. Click **Create**.

b. If you selected **URI** in the preceding step, in the **URI** field, enter the Uniform Resource Identifier (URI).

c. If you selected **OCI-compliant registry** in the preceding step, in the **OCI storage location** field, enter the URI.

10. Click **Add connection**.

Verification

- The connection that you added is displayed on the **Connections** tab for the project.

3.2. UPDATING A CONNECTION

You can edit the configuration of an existing connection as described in this procedure.

NOTE

Any changes that you make to a connection are not applied to dependent resources (for example, a workbench) until those resources are restarted, redeployed, or otherwise regenerated.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project, created a workbench, and you have defined a connection.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that contains the connection that you want to change.
   A project details page opens.
3. Click the **Connections** tab.
4. Click the action menu (⋮) beside the connection that you want to change and then click **Edit**.
   The **Edit connection** form opens.
5. Make your changes.
6. Click **Save**.

Verification

- The updated connection is displayed on the **Connections** tab for the project.

3.3. DELETING A CONNECTION

You can delete connections that are no longer relevant to your project.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project with a connection.

Procedure

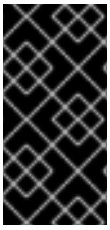
1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to delete the connection from.
   A project details page opens.
3. Click the **Connections** tab.
4. Click the action menu (⋮) beside the connection that you want to delete and then click **Delete connection**.
   The **Delete connection** dialog opens.
5. Enter the name of the connection in the text field to confirm that you intend to delete it.
6. Click **Delete connection**.

Verification

- The connection that you deleted is no longer displayed on the **Connections** page for the project.

3.4. USING THE CONNECTIONS API

You can use the connections API to enable flexible connection management to external data sources and services in OpenShift AI. Connections are stored as Kubernetes Secrets with standardized annotations that enable protocol-based validation and routing. Components use connections by referencing them in their resource specifications. The Operator and components use the **opendatahub.io/connections** annotation to establish the relationship between resources and connection Secrets.



IMPORTANT

For all new connection Secrets, use the annotation **opendatahub.io/connection-type-protocol**. The old annotation format, **opendatahub.io/connection-type-ref**, is deprecated. While both annotation formats are currently supported, **opendatahub.io/connection-type-protocol** takes precedence.

The connection API supports the following connection types:

- **s3**: S3-compatible object storage
- **uri**: Public HTTP/HTTPS URIs
- **oci**: OCI-compliant container registries

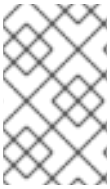
Additionally, the connection API supports the following workloads:

- Notebook (Workbenches)
- InferenceService (Model Serving)
- LLMInferenceService (llm-d Model Serving)

With connections API, the protocol-based annotation allows components to identify and validate appropriate connections for their use case. Protocol-specific integration logic is implemented by each component according to their needs.

All connections use the following basic structure:

```
apiVersion: v1
kind: Secret
metadata:
  name: <connection-name>
  namespace: <your-namespace>
  annotations:
    opendatahub.io/connection-type-protocol: "<protocol>" # Required: s3, uri, or oci
type: Opaque
data:
  # Protocol-specific fields (base64-encoded)
```



NOTE

While the Operator does minimal validation of **s3** and **uri** secrets, the overall correctness of the connection secret is the responsibility of the user. OpenShift AI might add more robust validation in future releases.

3.4.1. Namespace isolation in connections API

Connections are Kubernetes Secrets stored within user project namespaces. Cross-namespace access is not supported, which means that connections can only be used by resources within the same namespace where the connection Secret exists.

3.4.2. Role-based access control (RBAC) requirements in connections API

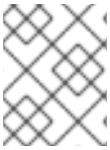
- You must have appropriate RBAC permissions to create, read, update, or delete Secrets in their project namespace.
- Components using connections must have ServiceAccount permissions to read Secrets in the namespace.
- Without access to a connection Secret, the using resource, for example, workbench or model serving, fails to start or function.

For more information about managing RBAC in OpenShift, see Using RBAC to define and apply permissions.

3.4.3. Validation scope

You can create connection Secrets with maximum flexibility, as the webhook validation for the Operator is advisory, not restrictive. With this flexibility, you can:

- Include the **opendatahub.io/connection-type-protocol** annotation to trigger validation of protocol-specific fields, which acts as helpful guidance.
- Create the Secret, even if you omit required annotations or include invalid fields; the system will not block secret creation.



NOTE

If your connection is invalid, it will cause workload failures at runtime. Always validate your connection credentials before deploying workloads.

3.4.4. Using connection annotations based on workload type

When configuring connections API, the format for referencing a connection Secret via the **opendatahub.io/connections** annotation changes based on the type of Kubernetes Custom Resource (CR) or workload being used.

- For a **Notebook** custom resource, multiple connections are supported. Use comma-separated values using the namespace/name format. For example: **opendatahub.io/connections: 'my-project/connection1,my-project/connection2'**.

- For **InferenceService** and **LLMInferenceService** custom resources, the Connection name is a simple string, assumed to be in the same namespace as the service. Only a single connection is supported. For example: **opendatahub.io/connections: 'my-connection'**.
- For **InferenceService** and **LLMInferenceService** using **S3 connections**, the additional annotation **opendatahub.io/connection-path** is used to specify the exact location of the model within the bucket. For example:

```
metadata:
annotations:
opendatahub.io/connections: "my-s3-connection"
opendatahub.io/connection-path: "my-bucket-path"  # Specify path within S3 bucket
```

3.4.5. Creating an Amazon S3-compatible connection type using the connections API

In OpenShift AI, you can create an Amazon S3-compatible connection type by using the connections API. In the following procedure, you define a Kubernetes Secret resource that holds the necessary credentials and configuration for an S3-compatible connection.

Prerequisites

- You have access to a Kubernetes cluster where you have permissions to create Secrets.
- You have the following details for your S3 storage: the S3 endpoint URL, bucket name, Access Key ID, and Secret Access Key.

Procedure

1. Create a YAML file (for example, **s3-connection.yaml**) that defines a Kubernetes **Secret** of type Opaque. This secret will contain the S3 connection parameters in the **stringData** section.

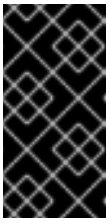
```
kind: Secret
metadata:
  name: <connection-name> # Choose a descriptive name for your connection
  namespace: <your-namespace> # Specify the namespace where the connection is needed
  annotations:
    opendatahub.io/connection-type-protocol: "s3"
type: Opaque
stringData:
  # --- REQUIRED FIELDS ---
  AWS_S3_ENDPOINT: "<s3-endpoint-url>" (1)
  AWS_S3_BUCKET: "<bucket-name>" (2)
  AWS_ACCESS_KEY_ID: "<access-key-id>" (3)
  AWS_SECRET_ACCESS_KEY: "<secret-access-key>" (4)
  # ----------------------

  # --- OPTIONAL FIELDS (Example) ---
  # AWS_DEFAULT_REGION: "us-east-1" (5)
```

   a. In the example YAML, replace the required fields by populating the placeholder values in the **stringData** section with your actual S3 connection details:

   (1) S3 endpoint URL: The full URL for your S3 compatible endpoint.

2. Mandatory bucket name: The exact name of the S3 bucket you intend to connect to.
3. Access key ID: Your S3 account access key ID.
4. Secret access key: Your S3 account secret access key.
5. Optional region field: If your S3 provider requires a specific region or if you are using AWS, you may include this optional field.



IMPORTANT

Configure credentials to access only the S3 resources for this project. Sharing AWS credentials between projects can lead to compromised credentials and unauthorized access to data. Configure IAM policies that grant only the minimum permissions required for the specific S3 bucket.

NOTE

The **opendatahub.io/connection-type-protocol: `s3`** annotation is required by applications to recognize this Secret as an S3 connection.

2. Apply the Secret to the cluster by using the **kubectl apply** command to create the Secret in your Kubernetes cluster.

```
kubectl apply -f s3-connection.yaml
```

3.4.5.1. Using an Amazon S3 connection with InferenceService custom resource

You can use an Amazon S3-compatible connection type with an **InferenceService** custom resource. In the following procedure, you define the storage location for your model when deploying a KServe **InferenceService** custom resource.

Prerequisites

- You have created an S3 connection Secret in the **project** namespace.
- You have deployed a KServe Operator in your cluster.
- Your model files are stored in the designated S3 bucket.

Procedure

1. Create a YAML file (for example, **inferenceservice.yaml**) that defines the KServe **InferenceService** custom resource. This resource defines how your model is served.
2. Specify the connection and path annotations in the **metadata.annotations** section.

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: my-model                # Name of the service
  namespace: my-project
  annotations:
```

```
    opendatahub.io/connections: "my-s3-connection"  1
    opendatahub.io/connection-path: "my-bucket-path"  2
spec:
  predictor:
    model:
      modelFormat:
        name: pytorch          # Specify the framework format (for example, pytorch, tensorflow)
      # NOTE: The storageUri will be automatically generated and injected here
      # by the operator (for example, storageUri: s3://my-bucket/my-bucket-path)
```

1. In the **opendatahub.io/connections** field, reference the name of your S3 connection Secret.
2. In the **opendatahub.io/connection-path** field, reference the folder path within the S3 bucket. This optional but highly recommended annotation specifies the path within the S3 bucket where your model files are located.

NOTE

When used with an **InferenceService** custom resource, the **opendatahub.io/connections** annotation usually requires the Secret name (for example, **my-s3-connection**) if the Secret is in the same namespace as the **InferenceService**.

3.4.5.2. Using an Amazon S3 connection with LLMInferenceService custom resource

You can use an Amazon S3-compatible connection type with the **LLMInferenceService** custom resource. In the following procedure, you define the storage location for your large language model (LLM) when deploying a KServe **LLMInferenceService** by using an S3 connection.

Prerequisites

- You have created an S3 connection Secret in the **project** namespace.
- You have deployed a KServe Operator that supports the **LLMInferenceService** custom resource.
- Your LLM model files are stored in the designated S3 bucket at a specific path.

Procedure

1. Create a YAML file (for example, **llm-service.yaml**) that defines the KServe **LLMInferenceService** custom resource. This resource is specialized for serving large language models.
2. Specify the connection and path annotations in the **metadata.annotations** section to link the service to your S3 storage.

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: my-llm-model              # Name of the LLM serving instance
  namespace: my-project
  annotations:
```

```
    opendatahub.io/connections: "my-s3-connection"      (1)
    opendatahub.io/connection-path: "my-bucket-path"    (2)
spec:
  model:
    # NOTE: The .spec.model.uri field is automatically injected by the operator
    # based on the connection and path annotations above.

    # Example of the injected field: .spec.model.uri: s3://my-bucket/my-bucket-path
```

1. In the **opendatahub.io/connections** field, reference the name of your S3 connection Secret. For example, **my-s3-connection**.
2. In the **opendatahub.io/connection-path** field, specify the path within the S3 bucket where your LLM model files are stored. For example, **my-bucket-path**.

3.4.6. Creating a URI-compatible connection type using the connections API

In OpenShift AI, you can create a URI-compatible connection type by using the connections API. In the following procedure, you define a Kubernetes Secret resource that holds a simple URI for connecting to an external resource, such as a model file hosted on an HTTP server or Hugging Face.

Prerequisites

- You have access to a Kubernetes cluster where you have permissions to create Secrets.
- You have access to the full HTTP/HTTPS URL or Hugging Face URI (**hf://**) for the target resource.

Procedure

1. Create a YAML file (for example, **uri-connection.yaml**) that defines a Kubernetes **Secret** of type **Opaque**. This secret will contain the URI in the **stringData** section.

```
apiVersion: v1
kind: Secret
metadata:
  name: <connection-name>
  namespace: <your-namespace>
  annotations:
    opendatahub.io/connection-type-protocol: "uri"
type: Opaque
stringData:
  URI: "<uniform-resource-identifier>" # The full URI/URL of the external resource
```

   a. In the example YAML, replace the required URI field by populating the placeholder value in the **stringData** section to include complete URL to the resource. This can be an HTTP/HTTPS link, or a Hugging Face URI.

NOTE

The **opendatahub.io/connection-type-protocol: uri** annotation is used by certain operators to identify the purpose of the Kubernetes Secret.

2. Apply the Secret to the cluster by using the **kubectl apply** command to create the Secret in your Kubernetes cluster.

```
kubectl apply -f uri-connection.yaml
```

3.4.6.1. Using a URI connection with InferenceService custom resource

You can use a URI-compatible connection type with an **InferenceService** custom resource. In the following procedure, you reference a pre-configured URI connection to define the storage location for your model when deploying a KServe **InferenceService**.

Prerequisites

- You have created a URI Connection Secret in the **project** namespace. For more information, see *Creating a URI connection type using the Connections API* .
- You have deployed a KServe Operator in your cluster.
- Your model file is accessible at the URI specified in the Secret.

Procedure

1. Create a YAML file (for example, **uri-inferenceservice.yaml**) that defines the KServe **InferenceService** custom resource.
2. Specify the URI connection annotation in the **metadata.annotations section**. Add the **opendatahub.io/connections** annotation and set its value to reference the URI Secret name, **my-uri-connection**.

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: my-model                # Name of the service
  namespace: my-project
  annotations:
    opendatahub.io/connections: "my-uri-connection"      # Reference to the URI Connection Secret
spec:
  predictor:
    model:
      modelFormat:
        name: sklearn           # Specify the framework format (for example, sklearn, tensorflow)
      # NOTE: The storageUri will be automatically generated and injected here
      # by the operator using the URI value from the Secret.
      # Example: .spec.predictor.model.storageUri: https://example.com/models/my-model.tar.gz
```

3. Apply the **InferenceService** custom resource by using the **kubectl apply** command.

```
kubectl apply -f uri-inferenceservice.yaml
```

3.4.6.2. Using a URI connection with LLMInferenceService custom resource

You can use a URI-compatible connection type with the **LLMInferenceService** custom resource. In the following procedure, you reference a pre-configured URI connection to define the storage location for your large language model (LLM) when deploying a KServe **LLMInferenceService**.

Prerequisites

- You have created a URI connection Secret in the **project** namespace.
- You have deployed a KServe Operator that supports the **LLMInferenceService** custom resource.
- Your LLM model files are accessible at the URI specified in the Secret.

Procedure

1. Create a YAML file (for example, **uri-llm-service.yaml**) that defines the KServe **LLMInferenceService** custom resource.
2. Specify the URI connection annotation in the **metadata.annotations** section. Add the **opendatahub.io/connections** annotation and set its value to reference the URI Secret name, **my-uri-connection**.

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: my-llm-model                # Name of the LLM serving instance
  namespace: my-project
  annotations:
    opendatahub.io/connections: "my-uri-connection"      # Reference to the URI Connection Secret
spec:
  model:
    # NOTE: The .spec.model.uri field is automatically generated and injected here
    # by the operator using the URI value from the Secret.
    # Example: .spec.model.uri: https://example.com/models/llm-model
```

3. Apply the **LLMInferenceService** custom resource by using the **kubectl apply** command.

```
kubectl apply -f uri-llm-service.yaml
```

3.4.7. Creating an OCI-compatible connection type using the connections API

In OpenShift AI, you can create an OCI-compatible connection type by using the connections API. In the following procedure, you define a Kubernetes Secret for storing credentials to an OCI-compatible container registry (like Quay.io or a private registry). This allows applications to authenticate and pull container images.

Prerequisites

- You have access to a Kubernetes cluster with permissions to create Secrets.
- You have access to the Registry URL with the organization (for example, http://quay.io/my-org).

- You have the username and password or token for the container registry.
- You have installed a tool for Base64 encoding (for example, base64 command-line utility).

Procedure

1. Prepare the authentication data by using Base64 to encode the **username:password** string, the **.dockerconfigjson** content, and the **OCI_HOST** URL.

   a. Encode credentials by combining **username:password** and encode it to get the value for the **auth** field in the JSON structure.

   ```
   echo -n 'myusername:mypassword' | base64
   # Result: <base64-encoded-username:password>
   ```

   b. Generate and encode **.dockerconfigjson** by creating the JSON structure and encoding the entire string with Base64.

   ```
   {
     "auths": {
       "quay.io": {
         "auth": "<base64-encoded-username:password>"
       }
     }
   }
   # The encoded result is: <base64-encoded-docker-config>
   ```

   c. Encode the full registry URL including the organization.

   ```
   echo -n 'http://quay.io/my-org' | base64
   # The encoded result is: <base64-encoded-registry-url>
   ```

2. Create a YAML file (for example, oci-connection.yaml) that defines a Kubernetes Secret of type **kubernetes.io/dockerconfigjson**. Use the encoded strings from the previous step in the **data** section.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: <connection-name>
     namespace: <your-namespace>
     annotations:
       opendatahub.io/connection-type-protocol: "oci" # Protocol identifier
   type: kubernetes.io/dockerconfigjson
   data:
     # Required Field: Base64-encoded Docker config JSON
     .dockerconfigjson: <base64-encoded-docker-config>

     # Required Field: Base64-encoded registry host URL with organization
     OCI_HOST: <base64-encoded-registry-url>
   ```

3. Apply the secret to the cluster by using the **kubectl apply** command to create the Secret.

   ```
   kubectl apply -f oci-connection.yaml
   ```

3.4.7.1. Using an OCI connection with InferenceService custom resource

You can use an OCI-compatible connection type with an **InferenceService** custom resource. In the following procedure, you define the private image registry location for your model by using an OCI connection when deploying a KServe **InferenceService** custom resource.

Prerequisites

- You have created an OCI connection Secret in the **project** namespace.
- You have deployed a KServe Operator in your cluster.

Procedure

1. Create a YAML file (for example, **oci-inferenceservice.yaml**) that defines the KServe **InferenceService** custom resource.
2. Specify the OCI connection annotation in the **metadata.annotations** section. Add the **opendatahub.io/connections** annotation and set its value to reference the OCI Secret name, **my-oci-connection**.
3. Define the model format by configuring the **.spec.predictor.model.modelFormat** field to specify the framework of the model (for example, **pytorch**).

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: my-model                # Name of the service
  namespace: my-project
  annotations:
    opendatahub.io/connections: "my-oci-connection"      # Reference to the OCI Connection Secret
spec:
  predictor:
    model:
      modelFormat:
        name: pytorch           # Specify the framework format (for example, pytorch)
      # NOTE: The operator webhook creates and injects .spec.predictor.imagePullSecrets
      # for OCI authentication based on the Secret.
```

4. Apply the **InferenceService** custom reource by using the **kubectl apply** command to create the resource.

```
kubectl apply -f oci-inferenceservice.yaml
```

3.4.7.2. Using an OCI connection with LLMInferenceService custom resource

You can use an OCI-compatible connection type with the **LLMInferenceService** custom resource. In the following procedure, you define the private image registry location for your Large Language Model (LLM) container image by using an OCI connection when deploying a KServe **LLMInferenceService**.

Prerequisites

- You have created an OCI connection Secret in the **project** namespace. For more information, see *Creating an OCI connection type using the Connections API* .
- You have a KServe Operator deployed that supports the **LLMInferenceService** custom resource.

Procedure

1. Create a YAML file (for example, **oci-llm-service.yaml**) that defines the KServe **LLMInferenceService** custom resource.
2. Specify the OCI connection annotation in the **metadata.annotations** section. Add the **opendatahub.io/connections** annotation and set its value to reference the OCI Secret name, **my-oci-connection**.

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: my-llm-model                # Name of the LLM serving instance
  namespace: my-project
  annotations:
    opendatahub.io/connections: "my-oci-connection"      # Reference to the OCI Connection Secret
spec:
  model:
    # Define the container image path here, if required.
    # NOTE: The operator webhook automatically injects `.spec.template.imagePullSecrets` for OCI authentication based on this connection.
    # The imagePullSecrets field will be set to the connection secret name.
```

3. Apply the **LLMInferenceService** custom resource by using the **kubectl apply** command.

```
kubectl apply -f oci-llm-service.yaml
```

CHAPTER 4. CONFIGURING CLUSTER STORAGE

4.1. ABOUT PERSISTENT STORAGE

OpenShift AI uses persistent storage to support workbenches, project data, and model training.

Persistent storage is provisioned through OpenShift storage classes and persistent volumes. Volume provisioning and data access are determined by access modes.

Understanding storage classes and access modes can help you choose the right storage for your use case and avoid potential risks when sharing data across multiple workbenches.

4.1.1. Storage classes in OpenShift AI

Storage classes in OpenShift AI are available from the underlying OpenShift cluster. A storage class defines how persistent volumes are provisioned, including which storage backend is used and what access modes the provisioned volumes can support. For more information, see Dynamic provisioning in the OpenShift documentation.

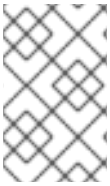
Cluster administrators create and configure storage classes in the OpenShift cluster. These storage classes provision persistent volumes that support one or more access modes, depending on the capabilities of the storage backend. OpenShift AI administrators then enable specific storage classes and access modes for use in OpenShift AI.

When adding cluster storage to your project or workbench, you can choose from any enabled storage classes and access modes.

4.1.2. Access modes

Storage classes create persistent volumes that can support different access modes, depending on the storage backend. Access modes control how a volume can be mounted and used by one or more workbenches. If a storage class allows more than one access mode, you can select the one that best fits your needs when you request storage. All persistent volumes support **ReadWriteOnce (RWO)** by default.

Access mode	Description
ReadWriteOnce (RWO) (Default)	The storage can be attached to a single workbench or pod at a time and is ideal for most individual workloads. **RWO** is always enabled by default and cannot be disabled by the administrator.
ReadWriteMany (RWX)	The storage can be attached to many workbenches simultaneously. **RWX** enables shared data access, but can introduce data risks.
ReadOnlyMany (ROX)	The storage can be attached to many workbenches as read-only. **ROX** is useful for sharing reference data without allowing changes.
ReadWriteOncePod (RWOP)	The storage can be attached to a single pod on a single node with read-write permissions. **RWOP** is similar to **RWO** but includes additional node-level restrictions.



NOTE

You can enable access modes that are required. A warning is displayed if you request an access mode with unknown support, but you can continue to select **Save** to create the storage class with the selected access mode.

4.1.2.1. Using shared storage (RWX)

The **ReadWriteMany (RWX)** access mode allows multiple workbenches to access and write to the same storage volume at the same time. Use **RWX** access mode for collaborative work where multiple users need to access shared datasets or project files.

However, shared storage introduces several risks:

- **Data corruption or data loss** If multiple workbenches modify the same part of a file simultaneously, the data can become corrupted or lost. Ensure your applications or workflows are designed to safely handle shared access, for example, by using file locking or database transactions.
- **Security and privacy**. If a workbench with access to shared storage is compromised, all data on that volume might be at risk. Only share sensitive data with trusted workbenches and users.

To use shared storage safely:

- Ensure that your tools or workflows are designed to work with shared storage and can manage simultaneous writes. For example, use databases or distributed data processing frameworks.
- Be cautious with changes. Deleting or editing files affects everyone who shares the volume.
- Back up your data regularly, which can help prevent data loss due to mistakes or misconfigurations.
- Limit access to RWX volumes to trusted users and secure workbenches.
- Use **ReadWriteMany (RWX)** only when collaboration on a shared volume is required. For most individual tasks, **ReadWriteOnce (RWO)** is ideal because only one workbench can write to the volume at a time.

4.2. ADDING CLUSTER STORAGE TO YOUR PROJECT

For projects that require data to be retained, you can add cluster storage to the project. Additionally, you can also connect cluster storage to a specific project's workbench.

Prerequisites

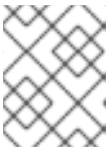
- You have logged in to Red Hat OpenShift AI.
- You have created a project that you can add cluster storage to.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to add the cluster storage to.

A project details page opens.

3. Click the **Cluster storage** tab.
4. Click **Add cluster storage**.
   The **Add cluster storage** dialog opens.
5. In the **Name** field, enter a unique name for the cluster storage.
6. Optional: In the **Description** field, enter a description for the cluster storage.
7. From the **Storage class** list, select the type of cluster storage.



NOTE

You cannot change the storage class after you add the cluster storage to the project.

8. For storage classes that support multiple access modes, select an **Access mode** to define how the volume can be accessed. For more information, see About persistent storage.
   Only the access modes that have been enabled for the storage class by your cluster and OpenShift AI administrators are visible.
9. In the **Persistent storage size** section, specify a new size in gibibytes or mebibytes.
10. Optional: If you want to connect the cluster storage to an existing workbench:
    a. In the **Workbench connections** section, click **Add workbench**.
    b. In the **Name** field, select an existing workbench from the list.
    c. In the **Path format** field, select **Standard** if your storage directory begins with **/opt/app-root/src**, otherwise select **Custom**.
    d. In the **Mount path** field, enter the path to a model or directory within a container where a volume is mounted and accessible. The path must consist of lowercase alphanumeric characters or **-**. Use / to indicate subdirectories.
11. Click **Add storage**.

Verification

- The cluster storage that you added is displayed on the **Cluster storage** tab for the project.
- A new persistent volume claim (PVC) is created with the storage size that you defined.
- The persistent volume claim (PVC) is visible as an attached storage on the **Workbenches** tab for the project.

4.3. UPDATING CLUSTER STORAGE

If your data science work requires you to change the identifying information of a project's cluster storage or the workbench that the storage is connected to, you can update your project's cluster storage to change these properties.

NOTE

You cannot directly change the **storage class** for cluster storage that is already configured for a workbench or project. To switch to a different storage class, you need to migrate your data to a new cluster storage instance that uses the required storage class. For more information, see Changing the storage class for an existing cluster storage instance.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project that contains cluster storage.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project whose storage you want to update.
   A project details page opens.
3. Click the **Cluster storage** tab.
4. Click the action menu (⋮) beside the storage that you want to update and then click **Edit storage**.
   The **Update cluster storage** page opens.
5. Optional: Edit the **Name** field to change the display name for your storage.
6. Optional: Edit the **Description** field to change the description of your storage.
7. Optional: In the **Persistent storage size** section, specify a new size in gibibytes or mebibytes. Note that you can only increase the storage size. Updating the storage size restarts the workbench and makes it unavailable for a period of time that is usually proportional to the size change.
8. Optional: If you want to connect the cluster storage to a different workbench:
   a. In the **Workbench connections** section, click **Add workbench**.
   b. In the **Name** field, select an existing workbench from the list.
   c. In the **Path format** field, select **Standard** if your storage directory begins with **/opt/app-root/src**, otherwise select **Custom**.
   d. In the **Mount path** field, enter the path to a model or directory within a container where a volume is mounted and accessible. The path must consist of lowercase alphanumeric characters or **-**. Use / to indicate subdirectories.
9. Click **Update storage**.

If you increased the storage size, the workbench restarts and is unavailable for a period of time that is usually proportional to the size change.

Verification

- The storage that you updated is displayed on the **Cluster storage** tab for the project.

4.4. CHANGING THE STORAGE CLASS FOR AN EXISTING CLUSTER STORAGE INSTANCE

When you create a workbench with cluster storage, the cluster storage is tied to a specific storage class. Later, if your data science work requires a different storage class, or if the current storage class has been deprecated, you cannot directly change the storage class on the existing cluster storage instance. Instead, you must migrate your data to a new cluster storage instance that uses the storage class that you want to use.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a workbench or project that contains cluster storage.

Procedure

1. Stop the workbench with the storage class that you want to change.
   a. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
   b. Click the name of the project with the cluster storage instance that uses the storage class you want to change.
   The project details page opens.
   c. Click the **Workbenches** tab.
   d. In the **Status** column for the relevant workbench, click **Stop**.
   Wait until the **Status** column for the relevant workbench changes from **Running** to **Stopped**.
2. Add a new cluster storage instance that uses the needed storage class.
   a. Click the **Cluster storage** tab.
   b. Click **Add cluster storage**.
   The **Add cluster storage** dialog opens.
   c. Enter a **name** for the cluster storage.
   d. Optional: Enter a **description** for the cluster storage.
   e. Select the needed **storage class** for the cluster storage.
   f. For storage classes that support multiple access modes, select an **Access mode** to define how the volume can be accessed. For more information, see About persistent storage.
   Only the access modes that have been enabled for the storage class by your cluster and OpenShift AI administrators are visible.
   g. Under **Persistent storage size**, enter a size in gibibytes or mebibytes.
   h. In the **Workbench connections** section, click **Add workbench**.

i. In the **Name** field, select an existing workbench from the list.

j. In the **Path format** field, select **Standard** if your storage directory begins with **/opt/app-root/src**, otherwise select **Custom**.

k. In the **Mount path** field, enter the path to a model or directory within a container where a volume is mounted and accessible. For example, **backup**.

l. Click **Add storage**.

3. Copy the data from the existing cluster storage instance to the new cluster storage instance.

a. Click the **Workbenches** tab.

b. In the **Status** column for the relevant workbench, click **Start**.

c. When the workbench status is **Running**, click **Open** to open the workbench.

d. In JupyterLab, click **File → New → Terminal**.

e. Copy the data to the new storage directory. Replace *<mount_folder_name>* with the storage directory of your new cluster storage instance.

```
rsync -avO --exclude='/opt/app-root/src/__<mount_folder_name>__' /opt/app-root/src/
/opt/app-root/src/__<mount_folder_name>__/
```

For example:

```
rsync -avO --exclude='/opt/app-root/src/backup' /opt/app-root/src/ /opt/app-
root/src/backup/
```

f. After the data has finished copying, log out of JupyterLab.

4. Stop the workbench.

a. Click the **Workbenches** tab.

b. In the **Status** column for the relevant workbench, click **Stop**.
Wait until the **Status** column for the relevant workbench changes from **Running** to **Stopped**.

5. Remove the original cluster storage instance from the workbench.

a. Click the **Cluster storage** tab.

b. Click the action menu (⋮) beside the existing cluster storage instance, and then click **Edit storage**.

c. Under **Existing connected workbenches**, remove the workbench.

d. Click **Update**.

6. Update the mount folder of the new cluster storage instance by removing it and re-adding it to the workbench.

a. On the **Cluster storage** tab, click the action menu (⋮) beside the new cluster storage instance, and then click **Edit storage**.

b. Under **Existing connected workbenches**, remove the workbench.

c. Click **Update**.

d. Click the **Workbenches** tab.

e. Click the action menu (⋮) beside the workbench and then click **Edit workbench**.

f. In the **Cluster storage** section, under **Use existing persistent storage**, select the new cluster storage instance.

g. Click **Update workbench.**

7. Restart the workbench.

a. Click the **Workbenches** tab.

b. In the **Status** column for the relevant workbench, click **Start**.

8. Optional: The initial cluster storage that uses the previous storage class is still visible on the **Cluster storage** tab. If you no longer need this cluster storage (for example, if the storage class is deprecated), you can delete it.

9. Optional: You can delete the mount folder of your new cluster storage instance (for example, the **backup** folder).

Verification

- On the **Cluster storage** tab for the project, the new cluster storage instance is displayed with the needed storage class in the **Storage class** column and the relevant workbench in the **Connected workbenches** column.
- On the **Workbenches** tab for the project, the new cluster storage instance is displayed for the workbench in the **Cluster storage** section and has the mount path: **/opt/app-root/src**.

4.5. DELETING CLUSTER STORAGE FROM A PROJECT

You can delete cluster storage from your projects to help you free up resources and delete unwanted storage space.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project with cluster storage.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.

2. Click the name of the project that you want to delete the storage from.
   A project details page opens.

3. Click the **Cluster storage** tab.

4. Click the action menu (⋮) beside the storage that you want to delete and then click **Delete storage**.
   The **Delete storage** dialog opens.

5. Enter the name of the storage in the text field to confirm that you intend to delete it.

6. Click **Delete storage**.

Verification

- The storage that you deleted is no longer displayed on the **Cluster storage** tab for the project.
- The persistent volume (PV) and persistent volume claim (PVC) associated with the cluster storage are both permanently deleted. This data is not recoverable.

CHAPTER 5. MANAGING ACCESS TO PROJECTS

5.1. CUSTOM ROLES FOR WORKBENCHES

You can use custom roles to control fine-grained, resource-level permissions for workbenches within a project. With custom roles, project administrators can define exactly who can view, edit, start, stop, or delete specific workbench instances, going beyond the default Admin and Contributor permission levels.

5.1.1. Kubernetes RBAC foundations

OpenShift AI custom roles build on the role-based access control (RBAC) model that is native to Kubernetes and OpenShift. In Kubernetes RBAC, access decisions are governed by three core objects:

Role

A **Role** defines a set of permissions as rules. Each rule specifies an API group, a resource type, and the verbs that are allowed on that resource. A **Role** is scoped to a single namespace, which maps to a single OpenShift AI project. A **ClusterRole** uses the same rules structure but is not namespace-scoped, allowing it to be reused across multiple projects.

RoleBinding

A **RoleBinding** grants the permissions defined in a **Role** or **ClusterRole** to a user, a group, or a service account within a namespace. A single role can be bound to multiple subjects, and a single subject can have multiple **RoleBindings**.

Subject

A subject is the user, group, or service account to which a **RoleBinding** grants permissions.

By using these standard Kubernetes objects, OpenShift AI custom roles inherit the security, auditability, and extensibility of the platform's existing access control framework. All permission changes are recorded by the Kubernetes API audit log, and roles integrate with your existing identity provider and group management without requiring a separate access control system.

5.1.2. Why fine-grained RBAC matters

The default Admin and Contributor permission levels in OpenShift AI projects provide broad access to all project resources. In environments where multiple users or teams share a project, these broad permission levels can be insufficient:

Least-privilege access

Organizations with security or compliance requirements must ensure that users have only the minimum permissions required for their tasks. A data scientist who runs preconfigured workbenches should not have permission to delete or reconfigure them.

Resource isolation

In shared projects, administrators might need to prevent users from viewing or interacting with workbenches that are not assigned to them, protecting sensitive configurations and data connections.

Operational safety

Restricting destructive actions, such as deleting workbenches, to a small set of administrators reduces the risk of accidental data loss in production environments.

Auditability

Fine-grained roles produce more meaningful audit trails. When each user's permissions are tightly scoped, audit logs clearly show which user had authorization to perform a specific action.

5.1.3. Workbenches and the Notebook custom resource

In OpenShift AI, each workbench is represented by a **Notebook** custom resource in the **kubeflow.org** API group. When you create, start, stop, or delete a workbench in the OpenShift AI dashboard, the dashboard performs the corresponding Kubernetes API operation on the **Notebook** resource in the project namespace. Custom roles for workbenches define permissions against this **Notebook** resource, which means that Kubernetes RBAC rules control access to workbench operations at the API level.

5.1.4. How custom roles work

Custom roles for workbenches are Kubernetes **Role** or **ClusterRole** objects that define permissions for **Notebook** resources. A **Role** applies to a single project namespace, while a **ClusterRole** can be reused across multiple projects. You create custom roles by using the **oc** CLI or by importing a YAML definition in the OpenShift web console. You then assign the roles to users or groups from the OpenShift AI dashboard.

To make a custom role visible in the OpenShift AI dashboard, you must apply the following label to the **Role** or **ClusterRole** object:

```
metadata:
  labels:
    opendatahub.io/dashboard: 'true'
```

Roles with this label appear in the dashboard's role selection list with an **AI role** label in the **Role type** column, distinguishing them from default OpenShift roles such as Admin or Edit.

5.1.5. Example use cases

The following examples illustrate how custom roles address common access control requirements:

Workbench reader

A user can view workbench configurations and status but cannot modify, start, or stop workbenches.

Workbench operator

A user can start and stop existing workbenches but cannot create new workbenches or edit their configurations.

Workbench editor

A user can create and modify workbenches but cannot delete them.

5.1.6. Scope and limitations

- Custom roles apply to workbench resources only. Granular RBAC for other project resources such as pipelines, model servers, and data connections is planned for a future release.
- You can only create and edit custom roles by using the **oc** CLI or by importing YAML in the OpenShift web console. Creating or editing custom roles directly in the OpenShift AI dashboard is not supported. A dashboard interface for administrators to create and configure custom roles is planned for a future release.
- A custom role must have the **opendatahub.io/dashboard: 'true'** label to display in the OpenShift AI dashboard. Roles without this label are not displayed.
- You can assign multiple custom roles to a single user or group. The effective permissions are the union of all assigned roles.

5.2. CREATE A CUSTOM WORKBENCH ROLE

You can create a custom Kubernetes role that defines fine-grained permissions for workbench resources in a project by using the **oc** CLI or by importing a YAML definition in the OpenShift web console. After you apply the required label, the role becomes available for assignment in the OpenShift AI dashboard.

NOTE

Creating or editing custom roles directly in the OpenShift AI dashboard is not supported. You must use the **oc** CLI or the OpenShift web console to create custom roles.

Prerequisites

- You have **cluster-admin** or project administrator privileges on the OpenShift cluster.
- You have access to the **oc** CLI or the OpenShift web console.

Procedure

1. Define a **Role** or **ClusterRole** object that specifies the permissions you want to grant for workbench resources. Use a **Role** to scope permissions to a single project namespace, or use a **ClusterRole** to define a reusable role that can be assigned across multiple projects. Save the following YAML to a file, or prepare it for import in the OpenShift web console.
   The following example creates a namespace-scoped role that allows a user to view workbenches but not modify them:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: <role_name>
  namespace: <project_namespace>
  labels:
    opendatahub.io/dashboard: true
rules:
  - apiGroups:
      - kubeflow.org
    resources:
      - notebooks
    verbs:
      - get
      - list
      - watch
```

   To create a **ClusterRole** instead, set **kind: ClusterRole** and omit the **namespace** field:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: <role_name>
  labels:
    opendatahub.io/dashboard: true
rules:
  - apiGroups:
```

```
  - kubeflow.org
resources:
  - notebooks
verbs:
  - get
  - list
  - watch
```

where:

_<role_name>_

Specifies a descriptive name for the role, such as **workbench-reader** or **workbench-operator**.

_<project_namespace>_

Specifies the namespace of the OpenShift AI project where the role applies. This field is required for a **Role** and must be omitted for a **ClusterRole**.

2. Create the role by using one of the following methods:

- **CLI**: Apply the YAML file by running the following command:

```
$ oc apply -f <role_file>.yaml
```

- **OpenShift web console**: In the web console, click the **Import YAML** icon (+) in the top navigation bar. Paste your YAML definition into the editor and click **Create**.

3. Verify that the role was created and that the required label is present:

```
$ oc get roles -n <project_namespace> <role_name> -o yaml
```

Confirm that the **opendatahub.io/dashboard: 'true'** label is output in the **metadata.labels** section. This label is required for the role to display in the OpenShift AI dashboard.

Verification

1. Log in to the OpenShift AI dashboard.
2. Navigate to the **Permissions** tab of the project where you created the role.
3. Click **Manage Permissions** and verify that your custom role appears in the role selection list with an **AI role** label in the **Role type** column.

Additional resources

- Section 5.7, “Custom workbench roles reference”
- Using RBAC to define and apply permissions in the OpenShift documentation

5.3. GRANT ACCESS TO A PROJECT

To enable your organization to work collaboratively, you can grant access to your project to other users and groups.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. From the list of projects, click the name of the project that you want to grant access to.
   A project details page opens.
3. Click the **Permissions** tab.
   The **Permissions** page for the project opens.
4. Click **Manage Permissions**.
5. From the **Subject Kind** list, select **User** or **Group**.
6. In the search field, enter the name of the user or group to whom you want to grant access. If the subject is not present in the list, enter the name directly.
7. From the role selection list, select one of the following access permission levels:
   - **Admin**: Subjects with this access level can edit project details and manage access to the project.
   - **Contributor**: Subjects with this access level can view and edit project components, such as its workbenches, connections, and storage.
8. Review the assignment summary in the confirmation dialog.
9. Click **Confirm** to apply the access permissions.
10. Optional: To grant access to additional users or groups, repeat this process.

Verification

- Users to whom you provided access to the project can perform only the actions permitted by their access permission level.
- The **Permissions** tab shows the users and groups that you granted access to and their assigned roles.

5.4. ASSIGN CUSTOM ROLES TO PROJECT USERS

As a project administrator, you can assign custom roles to users and groups from the **Permissions** tab in the OpenShift AI dashboard. Custom roles provide fine-grained, resource-level permissions for workbenches within a project.

Prerequisites

- You have logged in to Red Hat OpenShift AI.

- You have OpenShift AI administrator privileges or you are the project owner.
- You have created a project.
- One or more custom roles with the **opendatahub.io/dashboard: 'true'** label exist in the project namespace. For more information, see Section 5.2, “Create a custom workbench role” .

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project in which you want to assign custom roles.
   A project details page opens.
3. Click the **Permissions** tab.
   The **Permissions** page for the project opens.
4. Click **Manage Permissions**.
5. From the **Subject Kind** list, select **User** or **Group**.
6. In the search field, enter the name of the user or group to whom you want to assign a role. If the subject is not present in the list, enter the name directly.
7. From the role selection list, select the custom role to assign. The dashboard displays roles that have the **opendatahub.io/dashboard: 'true'** label with an **AI role** label in the **Role type** column, distinguishing them from default OpenShift roles.

NOTE

You can select multiple roles simultaneously for a single subject. The effective permissions are the union of all assigned roles.

8. Review the role assignment summary in the confirmation dialog.
9. Click **Confirm** to apply the role assignments.

Verification

- The **Permissions** tab shows the custom roles assigned to each user and group.
- Users with assigned custom roles can perform only the actions permitted by their roles.

5.5. UPDATE ACCESS TO A PROJECT

To change the level of collaboration on your project, you can update the access permissions of users and groups who have access to your project. You can manage permissions from the action menu on individual role assignments or by using the **Manage Permissions** flow.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have OpenShift AI administrator privileges or you are the project owner.

- You have created a project.
- You have previously assigned roles to users or groups in your project.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to change the access permissions of.
   A project details page opens.
3. Click the **Permissions** tab.
   The **Permissions** page for the project opens. Each row shows an individual role assignment for a user or group.
4. Locate the role assignment that you want to update and click the action menu (⋮).
5. Click **Manage permissions**.
   The **Manage Permissions** dialog opens.
6. From the role selection list, select the updated access permission level or custom role for the subject.
7. Review the updated assignment summary in the confirmation dialog.
8. Click **Confirm** to apply the updated access permissions.

Verification

- The **Permissions** tab shows the updated roles assigned to the users and groups whose access permissions you changed.

5.6. REMOVE ACCESS TO A PROJECT

You can restrict access to your project by unassigning roles from users and groups. Each role assignment is listed as a separate row on the **Permissions** tab, and you can unassign roles individually.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have OpenShift AI administrator privileges or you are the project owner.
- You have created a project.
- You have previously assigned roles to users or groups in your project.

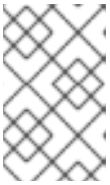
Procedure

1. From the OpenShift AI dashboard, click **Projects**.
   The **Projects** page opens.
2. Click the name of the project that you want to change the access permissions of.
   A project details page opens.

3. Click the **Permissions** tab.
   The **Permissions** page for the project opens. Each row shows an individual role assignment for a user or group.

4. Locate the role assignment that you want to remove and click the action menu (⋮).

5. Click **Unassign**.
   The **Unassign role?** dialog opens and warns that the user or group will lose the permissions that this role grants.



NOTE

If the role was assigned through OpenShift rather than through the OpenShift AI dashboard, the dialog warns that you cannot reassign the role from OpenShift AI. To reassign this role, you must use the OpenShift web console or CLI.

6. Click **Unassign role** to confirm.

Verification

- The role assignment no longer appears on the **Permissions** tab.
- The affected user or group can no longer perform the actions that the removed role permitted.

5.7. CUSTOM WORKBENCH ROLES REFERENCE

Custom workbench roles are Kubernetes **Role** or **ClusterRole** objects that define fine-grained permissions for workbench resources. A **Role** is scoped to a single project namespace, while a **ClusterRole** can be reused across multiple projects. This reference describes the label requirements, available resources, and verbs that you can use to build custom roles.

5.7.1. Label requirement

For a custom role to display in the OpenShift AI dashboard, you must apply the following label to the **Role** or **ClusterRole** object:

```
metadata:
  labels:
    opendatahub.io/dashboard: 'true'
```

Roles with this label are displayed with an **AI role** label in the **Role type** column in the dashboard, distinguishing them from default OpenShift roles.

5.7.2. Workbench resources and verbs

Each workbench in OpenShift AI is a **Notebook** custom resource in the **kubeflow.org** API group. The following table describes the Kubernetes resource and verbs that you can use in the **rules** section of your **Role** or **ClusterRole** definition to control workbench operations.

Table 5.1. Workbench resource verbs

API group	Resource	Available verbs
kubeflow.org	**notebooks**	**create**, **get**, **list**, **watch**, **update**, **patch**, **delete**

5.7.3. Example role definitions

The following examples show common custom role configurations for workbenches.

Workbench reader

Allows a user to view workbench configurations and status without making changes.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: workbench-reader
  labels:
    opendatahub.io/dashboard: 'true'
rules:
  - apiGroups:
      - kubeflow.org
    resources:
      - notebooks
    verbs:
      - get
      - list
      - watch
```

Workbench operator

Allows a user to start and stop existing workbenches without creating or deleting them.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: workbench-operator
  labels:
    opendatahub.io/dashboard: 'true'
rules:
  - apiGroups:
      - kubeflow.org
    resources:
      - notebooks
    verbs:
      - get
      - list
      - watch
      - patch
```

Workbench editor

Allows a user to create and modify workbenches without deleting them.

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
metadata:
  name: workbench-editor
  labels:
    opendatahub.io/dashboard: 'true'
rules:
  - apiGroups:
      - kubeflow.org
    resources:
      - notebooks
    verbs:
      - create
      - get
      - list
      - watch
      - update
      - patch
```

NOTE

The example roles above do not include the **delete** verb. To grant delete permissions for workbenches, add **delete** to the **verbs** list in your custom role's rules.

5.7.4. Default project roles

The following default roles remain available in the OpenShift AI dashboard alongside custom roles:

Table 5.2. Default project roles

Role	Permissions
Admin	Can edit project details and manage access to the project.
Contributor	Can view and edit project components, such as workbenches, connections, and storage.

Custom roles supplement these default roles. When a user has both a default role and one or more custom roles, the effective permissions are the union of all assigned roles.

CHAPTER 6. CREATING PROJECT-SCOPED RESOURCES FOR YOUR PROJECT

As an OpenShift AI user, you can access *global resources* in all OpenShift AI projects, but you can access *project-scoped resources* within the specified project only.

As a user with access permissions to a project, you can create the following types of project-scoped resources for your OpenShift AI project:

- Workbench images
- Model-serving runtimes for KServe
- Hardware profiles

All resource names must be unique within a project.

NOTE

A user with access permissions to a project can create project-scoped resources for that project, as described in Creating project-scoped resources.

Prerequisites

- You can access the OpenShift console.
- An OpenShift AI administrator has set the **disableProjectScoped** dashboard configuration option to **false**, as described in Customizing the dashboard.
- You can access a project in the OpenShift AI console.
- You have example YAML code for the type of resource that you want to create.
  You can get the YAML code from a trusted source, such as an existing project-scoped resource, a Git repository, or documentation. Alternatively, you can contact your cluster administrator to request the relevant YAML code.

Procedure

1. Log in to the OpenShift console.
2. From a trusted source, copy the YAML code that you want to use for your project resource.
   For example, if you can access an existing project-scoped resource in one of your projects, you can copy the YAML code as follows:
   a. In the **Administrator** perspective, click **Home → Search**.
   b. From the **Projects** list, select the appropriate project.
   c. In the **Resources** list, search for the relevant resource type, as follows:
      - For workbench images, search for **ImageStream**.
      - For hardware profiles, search for **HardwareProfile**.

- For serving runtimes, search for **Template**. From the resulting list, find the templates that have the **objects.kind** specification set to **ServingRuntime**.

d. Select a resource, and then click the **YAML** tab.

e. Copy the YAML content, and then click **Cancel**.

3. From the **Project** list, select your project name.
4. From the toolbar, click the **+** icon to open the **Import YAML** page.
5. Paste the example YAML content into the code area.
6. Edit the **metadata.namespace** value to specify the name of your project.
7. If necessary, edit the **metadata.name** value to ensure that the resource name is unique within the specified project.
8. Optional: Edit the resource name that is displayed in the OpenShift AI console, as follows:
   - For workbench images, edit the **metadata.annotations.opendatahub.io/notebook-image-name** value.
   - For hardware profiles, edit the **spec.displayName** value.
   - For serving runtimes, edit the **objects.metadata.annotations.openshift.io/display-name** value.
9. Click **Create**.

Verification

1. Log in to the OpenShift AI console.
2. Verify that the project-scoped resource is shown in the specified project:
   - For workbench images, when you create a workbench in the project, as described in Creating a workbench, the workbench image that you added is available in the **Image selection** list.
   - For model-serving runtimes, see Deploying models.
   - For workbench images, when you create a workbench in the project, as described in Creating a workbench, the workbench image that you added is available in the **Image selection** list.
   - For serving runtimes, see Deploying models.